- The response should have a `Content-Type` of `text/plain`.
- The response body should contain the `"pong"` string.

Each of these characteristics represents a different requirement, so you'd typically codify each as a separate unit test. This makes it easier to tell exactly which requirement hasn't been met when a test fails. For simplicity, in listing 36.3 I show all these assertions in the same test.

The positive case unit test is made more complex by the need to read the response body to confirm it contains `"pong"`. `DefaultHttpContext` uses `Stream.Null` for the `Response.Body` object, which means anything written to `Body` is lost. To capture the response and read it out to verify the contents, you must replace the `Body` with a `MemoryStream`. After the middleware executes, you can use a `StreamReader` to read the contents of the `MemoryStream` into a `string` and verify it.

Listing 36.3 Unit testing `StatusMiddleware` when a matching `Path` is provided

```
[Fact]
public async Task ReturnsPongBodyContent()
{
    var bodyStream = new MemoryStream();                          ❶
    var context = new DefaultHttpContext();                       ❶
    context.Response.Body = bodyStream;                           ❶
    context.Request.Path = "/ping";                               ❷
    RequestDelegate next = (ctx) => Task.CompletedTask;           ❸
    var middleware = new StatusMiddleware(next: next);            ❸

    await middleware.Invoke(context);                             ❹

    string response;                                              ❺
    bodyStream.Seek(0, SeekOrigin.Begin);                         ❺
```